

TBP Global Strategist

Technical Architecture Document

Version 1.2 — July 2026

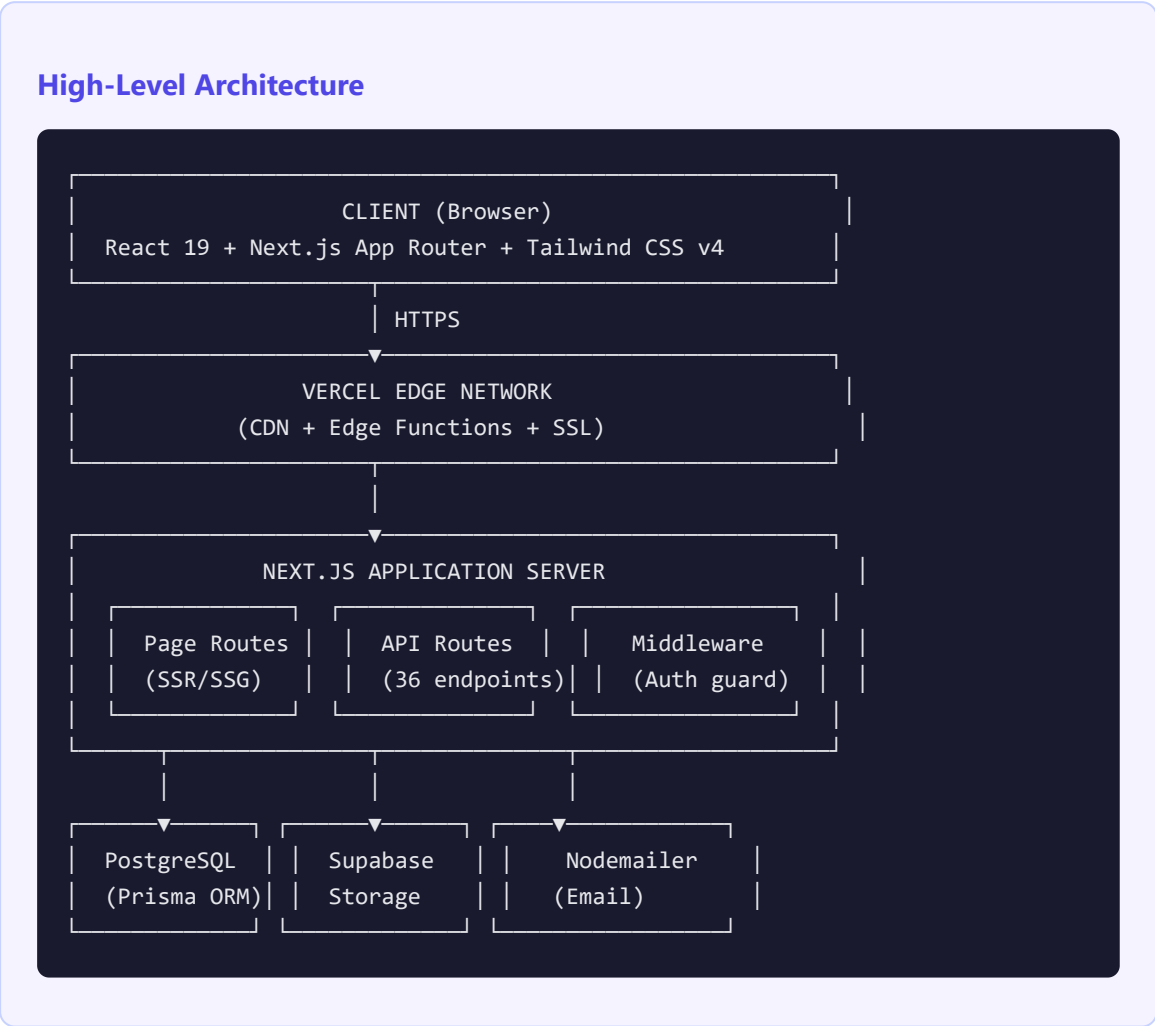
<https://tbpglobalstrategist.vercel.app>

Table of Contents

- 1. System Architecture Overview
- 2. Technology Stack
- 3. Application Architecture
- 4. Database Schema
- 5. Authentication & Authorization
- 6. API Architecture
- 7. Frontend Architecture
- 8. File Storage & Uploads
- 9. Deployment Architecture
- 10. Security Considerations
- 11. Performance & Optimization

1. System Architecture Overview

TBP Global Strategist is a full-stack web application built on **Next.js 16** with the App Router pattern. The system follows a monolithic architecture with server-side rendering (SSR), API routes, and static generation where appropriate.



2. Technology Stack

2.1 Frontend

Technology	Version	Purpose
Next.js	16.2.6	React framework with App Router, SSR, SSG, API routes
React	19.2.4	UI component library
TypeScript	5.x	Type-safe development
Tailwind CSS	4.x	Utility-first CSS framework

Framer Motion	12.x	Animations and transitions
Recharts	3.x	Chart and data visualization
Lucide React	1.16.0	Icon library

2.2 State Management

Technology	Version	Purpose
Zustand	5.x	Lightweight client-side state management
TanStack React Query	5.x	Server state management, caching, and data fetching
React Hook Form	7.x	Form state management with validation
Zod	4.x	Schema validation for forms and API inputs

2.3 Backend

Technology	Version	Purpose
Next.js API Routes	16.x	36 API endpoints under /api/*
Prisma	7.8.0	Type-safe database ORM
PostgreSQL	15+	Primary database (via Supabase)
NextAuth.js	5.0.0-beta.31	Authentication (Credentials + Google)
bcryptjs	3.x	Password hashing
Nodemailer	7.x	Transactional email sending

2.4 Infrastructure

Technology	Purpose
Vercel	Primary deployment platform (serverless functions, edge network)
Supabase	PostgreSQL database (PgBouncer on port 6543) + file storage
Docker	Alternative containerized deployment
PM2	Process manager for self-hosted deployment

3. Application Architecture

3.1 Route Groups

The Next.js App Router uses route groups to organize pages:

Route Group	Path	Purpose
(marketing)	/about, /contact, /insights, etc.	Public marketing pages with shared navbar/footer layout
(auth)	/login, /register	Authentication pages with split-screen layout
dashboard	/dashboard/*	Protected dashboard pages with sidebar layout
api	/api/*	36 REST API endpoints

3.2 Layout Hierarchy

```
RootLayout (src/app/layout.tsx)
├─ Providers (QueryClient, Theme, Toaster)
├─ Marketing Layout (navbar + footer + marketing chat)
│   ├── Home (/)
│   ├── About (/about)
│   ├── Strategists (/strategists)
│   ├── Projects (/projects)
│   └─ ... other marketing pages
├─ Auth Layout (split screen with TBP branding)
│   ├── Login (/login)
│   └─ Register (/register)
└─ Dashboard Layout (sidebar + header + breadcrumbs)
    ├── Dashboard Home (/dashboard)
    ├── Profile (/dashboard/profile)
    ├── Projects (/dashboard/projects)
    ├── Messages (/dashboard/messages)
    ├── Admin (/dashboard/admin)
    └─ ... other dashboard pages
```

3.3 Component Organization

Directory	Components	Purpose
components/ui/	20 components	Base UI primitives (shadcn/ui-style): button, card, dialog, sheet, tabs, table, etc.
components/layout/	7 components	Layout structure: navbar, footer, sidebar, providers, theme-provider

components/shared/	11 components	Reusable: animated-section, glass-card, gradient-text, search-input, video-modal, etc.
components/cards/	4 components	Card variants: strategist, project, organization, featured-project
components/chat/	2 components	Chat integration and marketing chat widget
components/strategist-profile-2/	10 components	Profile v2: hero, activity-timeline, project-grid, sidebar, etc.
components/admin/	3+ components	Admin dialogs: add-user, assign-areas, etc.

4. Database Schema

4.1 Database Configuration

Setting	Value
Provider	PostgreSQL (Supabase)
Connection Pooling	PgBouncer on port 6543
ORM	Prisma 7.8.0 with @prisma/adapters-pg
Client	Singleton pattern for development hot reload

4.2 Data Models (20 models)

Model	Purpose	Key Relationships
User	Core user entity with role, stage, and profile data	Has one StrategistProfile, many Projects, Messages, Notifications
Account	NextAuth OAuth account linking	Belongs to User
Session	Active user sessions	Belongs to User
VerificationToken	Email verification tokens	—
StrategistProfile	Extended profile with bio, stage, sector, category, location	Belongs to User, has many ExpertiseTags, UserWorkAreas
Organization	Company or organization entity	Has many OrganizationMembers
OrganizationMember	User-org membership with role	Belongs to User and Organization
Project	Core project with status, budget, dates, categories	Has many Milestones, Media, Contributors, Relations
Milestone	Project milestones with weight-based progress	Belongs to Project
ProjectMedia	Media attachments (images, videos, documents)	Belongs to Project
ProjectRelation	Relationships between projects	Belongs to Project

ProjectContributor	User-project membership	Belongs to User and Project
Tag	Tags for projects and expertise	Many-to-many with Projects and Profiles
ExpertiseTag	Profile-expertise tag junction	Belongs to StrategistProfile and Tag
Publication	Research publications	Belongs to User
Achievement	User achievements and certificates	Belongs to User
Notification	In-app notifications	Belongs to User
Message	Direct messages between users	Belongs to Conversation and User
Conversation	Chat conversations (1:1 and group)	Has many Messages and Participants
ConversationParticipant	Conversation membership	Belongs to Conversation and User
ActivityLog	Audit trail for user actions	Belongs to User
WorkArea	Work specialization areas (11 defaults)	Many-to-many with Users via UserWorkArea
UserWorkArea	User-work area assignments	Belongs to User and WorkArea
Submission	Document submissions for progression	Belongs to User

4.3 Enums

Enum	Values
Role	ADMIN, STRATEGIST, CORPORATE, ORGANIZATION_ADMIN, RESEARCHER, MODERATOR, PARTNER
ProjectStatus	DRAFT, ACTIVE, COMPLETED, CANCELLED
NotificationType	MESSAGE, PROJECT_INVITE, ORGANIZATION_INVITE, ACHIEVEMENT_UNLOCKED, PUBLICATION_APPROVED, SYSTEM
ProgressionStage	CANDIDATE, STRATEGIST, CONTRIBUTOR, PROJECT_ALIGNED, SECTOR_LEAD, PAID_ADVISER
ActivityAction	CREATE, UPDATE, DELETE, LOGIN, LOGOUT, JOIN_ORGANIZATION, LEAVE_ORGANIZATION, JOIN_PROJECT, LEAVE_PROJECT, PUBLISH

4.4 Migrations

Migration	Purpose
202507060001_init	Full schema creation (415 lines SQL)
202507070001_progression_stage	ProgressionStage enum + stage/sector columns
202507080001_location_fields	Added city, country, countryCode
202507080002_global_strategist_category	Added category field
202507080003_work_areas	Created WorkArea/UserWorkArea tables with 11 defaults
202507080004_submissions	Created Submission table

5. Authentication & Authorization

5.1 Authentication Flow

Credentials Provider:

1. User submits email + password via login form
2. NextAuth validates credentials against User table
3. Password verified using bcryptjs
4. JWT session token created and stored in cookie
5. User redirected to dashboard

Google Provider:

1. User clicks "Continue with Google"
2. Redirected to Google OAuth consent screen
3. Google returns authorization code
4. NextAuth exchanges code for tokens
5. User profile created/linked via PrismaAdapter
6. JWT session token created

5.2 Authorization Rules

Route Pattern	Requirement
/dashboard/*	Authenticated (any role)
/dashboard/admin/*	Authenticated + ADMIN role
/api/profile/*	Authenticated
/api/projects (POST)	ADMIN role
/api/admin/*	ADMIN role
/api/projects (GET)	Public
/api/strategists	Public
/login , /register	Public (redirect if already authenticated)

5.3 Session Strategy

- **Strategy:** JWT (JSON Web Tokens)
- **Storage:** HTTP-only cookies

- **Adapter:** PrismaAdapter for session/account storage in database
- **Trust Host:** Enabled for Vercel deployment

6. API Architecture

6.1 API Endpoints Summary

Category	Endpoints	Methods
Authentication	<code>/api/auth/[...nextauth]</code> , <code>/api/register</code> , <code>/api/seed</code>	GET, POST
Profile	<code>/api/profile</code> , <code>/api/profile/featured-project</code>	GET, PATCH, PUT
Strategists	<code>/api/strategists</code>	GET
Projects	<code>/api/projects</code> , <code>/api/projects/[id]</code> , <code>/api/projects/featured</code> , <code>/api/projects/categories</code> , <code>/api/projects/search</code>	GET, POST, PATCH, DELETE
Project Sub-resources	<code>/api/projects/[id]/milestones</code> , <code>/api/projects/[id]/media</code> , <code>/api/projects/[id]/relations</code> , <code>/api/projects/[id]/join</code> , <code>/api/projects/[id]/feature</code>	GET, POST, DELETE, PATCH
Messages	<code>/api/messages</code> , <code>/api/messages/[conversationId]</code>	GET, POST, PATCH
Notifications	<code>/api/notifications</code>	GET, PATCH
Submissions	<code>/api/submissions</code> , <code>/api/submissions/download</code>	GET, POST
File Upload	<code>/api/upload</code> , <code>/api/uploads</code>	POST, GET
Dashboard	<code>/api/dashboard/stats</code> , <code>/api/dashboard/activities</code> , <code>/api/dashboard/collaborations</code> , <code>/api/dashboard/my-projects</code>	GET
Admin	<code>/api/admin/users</code> , <code>/api/admin/users/[id]/*</code> , <code>/api/admin/work-areas</code> , <code>/api/admin/migrate-files</code>	GET, POST, PATCH, DELETE

6.2 API Response Format

```
// Success
{ "data": [...], "pagination": { "page": 1, "limit": 20, "total": 100 } }

// Error
{ "error": "Error message", "status": 400 }
```

```
// Authentication
{ "user": { "id": "...", "name": "...", "email": "...", "role": "STRATEGIST" } }
```

6.3 File Upload Flow

1. Client sends file via multipart form to `/api/upload`
2. Server validates file type and size
3. File uploaded to Supabase Storage bucket
4. Public URL returned to client
5. URL stored in database (ProjectMedia, Submission, etc.)

7. Frontend Architecture

7.1 State Management Strategy

Layer	Technology	Usage
Server State	TanStack React Query	API data fetching, caching, optimistic updates
Client State	Zustand	UI state, user preferences, shopping cart (future)
Form State	React Hook Form	Form inputs, validation, submission
URL State	Next.js searchParams	Filters, pagination, search queries

7.2 Component Patterns

- **Server Components:** Layout files, static pages, API route handlers
- **Client Components:** Interactive UI, forms, modals, real-time features (marked with "use client")
- **Compound Components:** Dialog, Sheet, DropdownMenu (shadcn/ui pattern)
- **Render Props:** Used in custom hooks for flexible component composition

7.3 Styling System

- **Tailwind CSS v4:** Utility-first styling with CSS custom properties for theming
- **Dark Mode:** CSS variable-based theme switching via ThemeProvider
- **Glass Morphism:** Custom GlassCard component with backdrop-blur effects
- **Gradients:** Reusable gradient classes for cards, buttons, and backgrounds
- **Animations:** Framer Motion for page transitions, scroll animations, and micro-interactions

7.4 Key UI Components

Component	Purpose	Lines
HeroSection	Landing page hero with video modal CTA	~140
DashboardLayout	Sidebar + header + breadcrumbs + notifications	289
VideoModal	Responsive Vimeo video embed modal	~70
GlassCard	Glass morphism card with hover effects	~30
AnimatedSection	Scroll-triggered animation wrapper	~25
ProgressionPathway	6-stage progression visualization	~200

8. File Storage & Uploads

8.1 Storage Architecture

Aspect	Details
Provider	Supabase Storage
Access	Service role key for server-side operations
Public Access	Uploaded files are publicly accessible via URL
File Types	Images (JPEG, PNG, WebP), Documents (PDF, DOCX, PPTX, XLSX, CSV, TXT, RTF, ODT, ODS)

8.2 Upload Flow

1. Client selects file via ImageUpload component or submission form
2. File sent to `/api/upload` endpoint
3. Server uploads to Supabase Storage bucket
4. Public URL returned and stored in database
5. URL used in ProjectMedia, Submission, or profile avatar

8.3 Static Assets

Directory	Contents
<code>public/images/</code>	8 project category images (JPEG, PNG, WebP)
<code>public/docs/</code>	PDF documentation (Overview, User Guide, Technical Architecture)
<code>public/uploads/</code>	User-uploaded files

9. Deployment Architecture

9.1 Vercel Deployment

Setting	Value
Platform	Vercel
Build Command	<code>prisma generate && next build --webpack</code>

Framework	Next.js 16
Node Version	24.x
Region	Washington, D.C. (iad1)
Production URL	https://tbpglobalstrategist.vercel.app

9.2 Build Process

1. `prisma generate` — Generates Prisma Client from schema
2. `next build --webpack` — Compiles TypeScript, bundles assets, generates static pages
3. Static pages pre-rendered at build time
4. Dynamic pages server-rendered on demand
5. API routes deployed as serverless functions

9.3 Alternative Deployments

Docker

```
# Build
docker build -t tbpglobalstrategist .

# Run
docker run -p 3000:3000 tbpglobalstrategist
```

PM2 (Self-Hosted)

```
# Build
npm run build

# Start with PM2
pm2 start npm --name "tbp" -- start
```


10. Security Considerations

10.1 Authentication Security

- Passwords hashed with bcryptjs (salt rounds: 12)
- JWT tokens stored in HTTP-only cookies
- Session invalidation on logout
- CSRF protection via NextAuth

10.2 API Security

- Route protection via NextAuth middleware
- Role-based access control (RBAC) for admin routes
- Input validation with Zod schemas
- SQL injection prevention via Prisma ORM

10.3 Data Security

- Environment variables for sensitive configuration
- Supabase RLS (Row Level Security) policies
- No secrets committed to git repository
- .gitignore excludes .env, node_modules, .next

10.4 Frontend Security

- Content Security Policy headers (via Vercel)
- XSS prevention via React's automatic escaping
- HTTPS enforced in production
- Secure cookie attributes (HttpOnly, Secure, SameSite)

11. Performance & Optimization

11.1 Rendering Strategy

Strategy	Usage
Static Generation (SSG)	Marketing pages, public profiles, project listings
Server-Side Rendering (SSR)	Dashboard pages, admin panels, dynamic content
Client-Side Rendering	Interactive components, real-time features, modals

11.2 Optimization Techniques

- **Code Splitting:** Automatic with Next.js App Router
- **Image Optimization:** Custom image-optimize.ts for client-side optimization
- **Tailwind Purging:** Automatic unused CSS removal
- **React Query Caching:** Stale-while-revalidate strategy for API data
- **Bundle Analysis:** Next.js built-in bundle analyzer

11.3 Database Optimization

- **Connection Pooling:** PgBouncer for efficient connection management
- **Indexed Queries:** Prisma indexes on frequently queried fields
- **Selective Fields:** Prisma select for fetching only needed data
- **Pagination:** Cursor-based and offset pagination for large datasets